

Estructuras de información

Recursividad

- Un método recursivo se invoca a sí mismo
- La recursividad normalmente produce un código más corto.
- En general, los algoritmos recursivos son muy elegantes
- Consta de 2 partes: Caso base y dominio
- El ejemplo canónico de un método recursivo es el factorial.
- En este ejemplo:
 $n! = n \cdot (n-1)!$ → Dominio (caso recursivo)
 $1! = 1$ → Caso base

```
class Recursion
{
    public static int factorial(int n)
    {
        if(n <= 1) return 1;
        else return n * factorial(n-1);
    }

    public static void main(String args[])
    {
        System.out.println(factorial(5));
    }
}
```

Características

- 1) División del problema → Problema grande se divide en subproblemas
- 2) Regresión → Después de alcanzar el caso base, las soluciones de los subproblemas se combinan para resolver el problema original a medida que las funciones recursivas retornan sus resultados.
- 3) Stack (pila de llamadas) → Cada llamada recursiva va a la pila de llamadas hasta que alcanza su caso base. Luego la pila se “desenrolla” resolviendo cada problema en orden inverso.

Ejemplo conceptual

Imagina que estás en un laberinto y quieres encontrar la salida. Una estrategia recursiva podría ser:

1. Si estás en la salida, detente (caso base).
2. Si no, elige una dirección y avanza.
3. Si te encuentras en un callejón sin salida, retrocede (esta sería la parte donde "desenrollas" la recursión).
4. Repítelo hasta encontrar la salida.

Aquí, cada movimiento en el laberinto es una llamada recursiva que explora un camino hasta que se alcanza el caso base (encontrar la salida o no tener más caminos que explorar).

Tipos de recursividad

- Recursividad Directa: Una función se llama a sí misma directamente.

Ejemplo: El cálculo del factorial (factorial(n) llama a factorial(n-1)).

- Recursividad Indirecta: Una función llama a otra función, que eventualmente vuelve a llamar a la función original.

```
void A() { B();  
}
```

```
void B() { A();  
}
```

- * Recursividad Simple: La función se llama a sí misma una vez en su cuerpo.

Ejemplo: El cálculo del factorial.

- Recursividad Múltiple: La función se llama a sí misma varias veces.

Ejemplo: El cálculo de la serie de Fibonacci, donde fibonacci(n) llama a fibonacci(n-1) y fibonacci(n-2).

Recursividad - Factorial

Una solución iterativa:

```
public static int factorial(int n)
{
    int result = 1 ;

    for ( int i = 1; i <= n; i ++ )
        result = result * i ;

    return result ;
}
```

The diagram shows the recursive steps for calculating factorial(5). It starts with 'factorial(5)' and expands into '5 * factorial(4)', then '5 * 4 * factorial(3)', '5 * 4 * 3 * factorial(2)', and finally '5 * 4 * 3 * 2 * factorial(1)'. The 'factorial(1)' is highlighted as the 'base step'. The process then contracts back up: '5 * 4 * 3 * 2 * 1' becomes '5 * 4 * 6', then '5 * 24', and finally '120'. Arrows labeled 'expansion' and 'contraction' indicate the direction of the process.

```
factorial(5)
5 * factorial(4)
5 * 4 * factorial(3)
5 * 4 * 3 * factorial(2)
5 * 4 * 3 * 2 * factorial(1)
5 * 4 * 3 * 2 * 1
5 * 4 * 3 * 2
5 * 4 * 6
5 * 24
120
```

expansion

base step

contraction

Sucesión Fibonacci

La sucesión fue descrita por Fibonacci como la solución a un problema de la cría de conejos

“Cierta persona tenía una pareja de conejos en un lugar cerrado y deseaba saber cuántos conejos se podrían reproducir en un año a partir de la pareja inicial, teniendo en cuenta que de forma natural tienen una pareja en un mes, y que a partir del segundo mes se empiezan a reproducir”.



Sucesión Fibonacci

La serie de Fibonacci comienza con 1 y 1, y tiene la propiedad de que cada Fibonacci posterior es la suma de los dos Fibonacci anteriores.

Digamos que $\text{fib}(0) = 1$, $\text{fib}(1) = 1$, $\text{fib}(2) = 2$, $\text{fib}(3) = 3$, $\text{fib}(4) = 5$, $\text{fib}(5) = 8$, . . .

NOTA

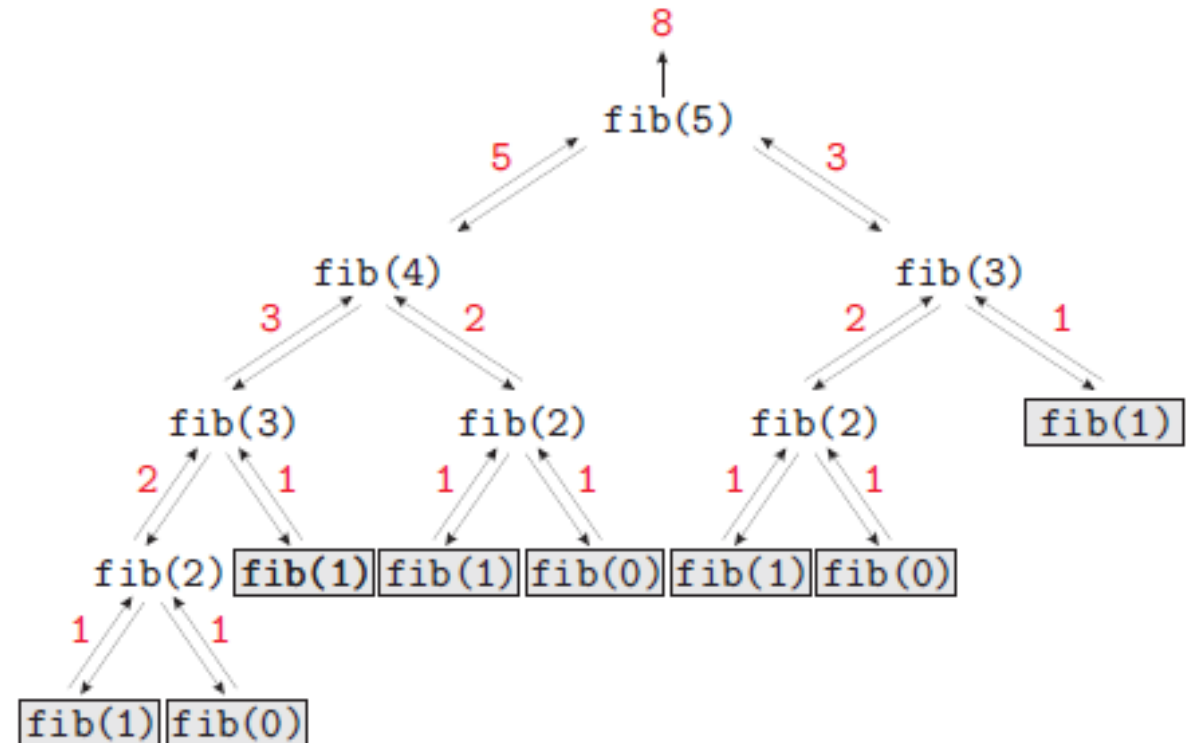
$\text{fib}(n) = \text{fib}(n - 1) + \text{fib}(n - 2)$ (Dominio)

$\text{fib}(0) = 1$, $\text{fib}(1) = 1$ (Caso base)

Sucesión Fibonacci

¿Cuántas veces invocamos el método Fibonacci al calcular el fib (5)?

```
public static int fib(int n)
{
    if (n <= 1) return 1;
    else return fib(n-1) + fib(n-2);
}
```



Recursividad

PROS:

- * Legibilidad: El código es más fácil de entender.
- * Facilita solución de problemas.
- * Facilita el mantenimiento, más fáciles de modificar.

Contras:

- * Consumo de memoria: cada llamada recursiva al sistema agrega nueva entrada en la pila de llamadas.
- * Eficiencia: pueden ser menos eficiente en términos de tiempo de ejecución

Ciclo

PROS:

- * Eficiencia: más eficiente en términos de uso de memoria y tiempo de ejecución.
- * Proporciona control directo sobre el flujo del programa, puede ser más adecuado cuando se requiere un control preciso sobre la iteración.

Contras:

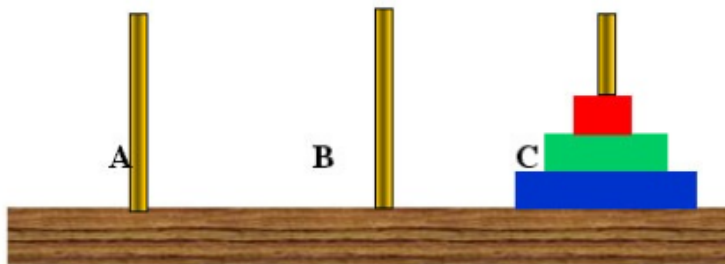
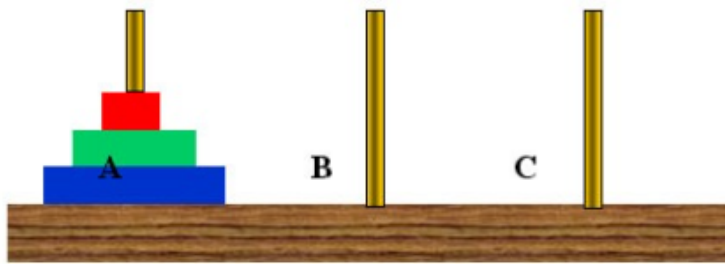
- * Legibilidad
- * Mayor complejidad: Especialmente si hay estructuras anidadas o múltiples condiciones.

¿Cuándo usar la recursividad?

Estas no son reglas precisas, sino más bien pautas:

- Problemas que se pueden dividir en subproblemas más pequeños
- Algoritmos de búsqueda y recorrido
- Problemas matemáticos

Torres de Hanoi



Algoritmo Torres de Hanói (Complejidad $\Theta(2^n - 1)$)

Entrada: Tres pilas de números *origen*, *auxiliar*, *destino*, con la pila *origen* ordenada

Salida: La pila *destino*

1. **si** *origen* == {1} **entonces**

1. **mover** el disco 1 de pila origen a la pila destino (insertarlo arriba de la pila destino)

2. **terminar**

2. **si no**

1. *hanoi*({1, ..., n-1}, *origen*, *destino*, *auxiliar*) //mover todas las fichas menos la más grande (*n*) a la varilla auxiliar

3. **mover** disco *n* a *destino* //mover la ficha grande hasta la varilla final

4. *hanoi* (*auxiliar*, *origen*, *destino*) //mover todas las fichas restantes, 1...n-1, encima de la ficha grande (*n*)

5. **terminar**

Ventajas y desventajas de la recursividad

- **Simplicidad y Claridad:** Para problemas que se pueden dividir en subproblemas similares, la recursividad puede ofrecer una solución muy limpia y directa.
- **Soluciones Elegantes:** Muchos problemas, como la travesía de árboles o la generación de combinaciones/permutaciones, se resuelven de manera elegante usando recursividad.
- **Facilidad para Problemas Recursivos:** Algunos problemas, como aquellos que se definen naturalmente de manera recursiva (por ejemplo, problemas de fractales, DFS en grafos), se implementan más fácilmente con recursividad.

Desventajas de la Recursividad

- **Costos de Memoria:** La recursión puede ser menos eficiente que la iteración debido al almacenamiento de múltiples llamadas en la pila.
- **Riesgo de Stack Overflow:** Si la recursión es profunda, puede llevar a un desbordamiento de la pila (stack overflow), especialmente en lenguajes que no optimizan la recursión de cola.
- **Complejidad en Depuración:** Depurar código recursivo puede ser más difícil que depurar código iterativo debido a la naturaleza anidada de las llamadas.